

Xray-quicklink

Fork of Xray-core — Performance & Feature Comparison

What is this?

Xray-quicklink is a fork of **Xray-core** (the proxy engine behind Xray) built to solve one specific problem: **streaming video through a multi-hop proxy chain without stalling**. Standard Xray-core kills QUIC/HTTP3 video sessions after 2 minutes of inactivity, randomly flips between IPv4 and IPv6 across connections, and creates a new outbound socket for every flow — all of which cause YouTube, Hulu, Disney+, and Netflix to stall every few minutes.

The fork adds a UDP socket pool, a sticky DNS resolver, QUIC connection migration tracking, a TCP warm-pool with automatic pre-warming, DNS prefetch, and dozens of performance optimizations that make proxying video feel as smooth as a direct connection. This document compares the fork against upstream Xray-core.

QUIC / UDP Performance

Path	Upstream	This Fork	Speedup
First QUIC Initial (cold sniff)	~13,000 ns 12 allocs, 32 KB	521 ns 3 allocs, 613 B	25x faster
Initial retransmit (cached keys)	~13,000 ns 12 allocs	134 ns 2 allocs, 56 B	97x faster
1-RTT short header (99% of traffic)	~1,200 ns 1 alloc (~1.2 KB)	2.1 ns 0 allocs, 0 B	570x faster
Inbound worker (per packet)	~200 ns 4 allocs	~60 ns 0 allocs	3x faster
UDP socket creation	1 socket per QUIC flow	1 socket per destination IP	N x fewer FDs
Reply demux	N/A (no pool)	RLock + struct key zero contention	Lock-free
Packet buffer allocs	make() per reply	sync.Pool 0 steady-state allocs	0 allocs

The 1-RTT speedup is the most important: 99% of packets in a long-lived QUIC connection (e.g., a YouTube video stream) are 1-RTT short-header packets. Upstream clones every one; this fork returns after a single byte test.

TCP Performance & Reliability

Feature	Upstream	This Fork	Impact
TCP connection reuse	1 outbound per inbound (no reuse)	Warm pool: outbound kept alive, reused by next inbound	Eliminates TCP+TLS handshake
TCP pre-warming	None	Learns critical-path IPs per site family, pre-warms every 60s	2x faster first visit
Shared-IP dedup	N/A	Domains sharing an IP share one warm connection	Fewer connections needed
CLOSE-WAIT cleanup	30 min linger (socket sits idle)	Immediate (inputCloser propagates EOF)	No stale sockets
Keepalive config	tcpKeepAlive silently ignored (wrong field name)	tcpKeepAlive aliased to tcpKeepAliveIdle (works)	Dead conns detected in 15s, not never
TCP user timeout	Kernel default (~15 min)	Configurable (30s recommended)	Survives WG blips
Pipe buffer	16 KB	256 KB	Absorbs YouTube bursts
IPv6 to IPv4 fallback	None	Automatic on ENETUNREACH	Broken IPv6 doesn't stall connections

WireGuard Integration

The fork doesn't use Xray's built-in WireGuard protocol (userspace wireguard-go). Instead, it uses the **freedom** outbound with **sockopt.interface** to bind to a kernel-space WireGuard interface. This gives you kernel-accelerated ChaCha20-Poly1305 (SIMD on ARM NEON / x86 AVX2) and the UDP socket pool applies to QUIC traffic through the tunnel.

Feature	Upstream	This Fork
TCP through WG	Works (interface + SO_MARK)	Same + warm pool eliminates handshake (200-600ms saved)
Pooled QUIC through WG	No pool = N/A	Pool sockets honor SO_BINDTODEVICE + SO_MARK (v26.10.17)
DNS for WG geo-domains	net.DefaultResolver (bypasses xray DNS config)	internet.LookupForIP (honors dns.servers domain rules)

DNS prefetch for WG	None	Periodic goroutine keeps WG-routed domains cached
TCP pre-warm through WG	None	Auto-learns critical-path IPs and pre-establishes TCP connections

What Makes This Fork Unique

1-RTT Zero-Overhead Sniffer

The fork's QUIC sniffer returns immediately for 99% of packets (1-RTT short headers) with a single byte test — zero allocations, zero mutex acquires. Upstream Xray-core clones the entire packet buffer on every single packet, even ones it immediately rejects. At 1000+ packets/sec, this is the difference between invisible CPU overhead and measurable garbage collection pressure.

DCID-Keyed Sniffer Cache

The fork caches HKDF-derived keys and extracted SNI per QUIC DCID. When a QUIC Initial retransmits (common on lossy multi-hop chains), the cached keys are reused — skipping the full HKDF + AES-GCM derivation. This is 97x faster than upstream's re-derive-every-time approach. No other proxy does this.

UDP Socket Pool with DCID Demux

One outbound UDP socket per destination IP, shared across all inbound QUIC sessions. Reply packets are demuxed by QUIC Connection ID (DCID), so 50+ concurrent YouTube sessions to the same CDN edge share one socket instead of creating 50 separate ones. Upstream creates a new socket per flow — at 50 flows that's 50 FDs, 50 kernel sockets, and 50 separate source ports.

QUIC Connection Migration Tracking

When a mobile client's NAT binding changes (source port flip) or QUIC rotates its Connection ID (NEW_CONNECTION_ID frames, common on Android YouTube), the fork tracks the migration and keeps the existing connection alive. Upstream Xray-core keys sessions by 4-tuple (source IP:port + dest IP:port) and kills the session on any change — causing the video to stall until a new connection is established.

TCP Warm Pool with First-N Pre-Warming

When a TCP connection's inbound side closes, the outbound is kept alive in a warm pool for reuse by the next inbound to the same destination. The pool learns which IPs are hit first on each site visit (the critical-path IPs) and pre-establishes TCP connections to them every 60 seconds. When you switch from Hulu to Disney through a WireGuard tunnel, Disney's TCP connection is already open — saving 200-600ms of handshake through the tunnel. Domains sharing an IP (e.g., hulu.com and asset.hulu.com on the same CDN edge) automatically share one warm connection.

Sticky DNS Resolver with Wildcard Cache

Caches DNS resolutions per hostname with a configurable TTL (default 60s for YouTube). When the TTL expires, serves the stale IP immediately while refreshing in the background (stale-while-revalidate). Includes a wildcard cache (*.googlevideo.com) that covers all subdomains of a service, with iterative label stripping for deep subdomains. When the resolver detects a CDN edge rotation, it notifies the UDP pool to invalidate sockets to the old IP — closing

the gap between DNS TTL and pool staleness.

DNS Prefetch

A periodic goroutine scans the DNS cache every 60 seconds and refreshes entries whose A or AAAA record expires within 30 seconds. Combined with `serveStale`, the cache is always fresh — no DNS-lookup-induced stalls. The sticky resolver uses Xray's own DNS client (not Go's `stdlib`), so it honors per-domain routing rules (e.g., `geosite:amazon` through a US-geo DNS server for WireGuard).

Architecture Comparison

Feature	Upstream	This Fork
QUIC connection migration	None (4-tuple keyed)	DCID-keyed migration
UDP session timeout	2 min (kills video)	30 min (configurable)
Sticky DNS resolver	None (re-resolves per flow)	Per-hostname + wildcard + stale-while-revalidate
DNS prefetch	None	prefetchInterval + prefetchThreshold
Multi-IP inbound listen	Single IP per inbound	Array of IPs, one handler per IP
Source IP binding	Not supported	sendThrough: origin
SIGHUP hot reload	Full restart required	Routing rules reloaded atomically
Sniffer features	SNI only	SNI + ALPN + ECH detection
TCP warm-pool	None	Outbound TCP reused after close
TCP pre-warming	None	First-N learning + shared-IP dedup
IPv6 to IPv4 fallback	None	Automatic on ENETUNREACH

Summary

The fork transforms Xray-core from a general-purpose L4 proxy into a **video-streaming-optimized proxy** that handles QUIC/HTTP3 through multi-hop chains and WireGuard tunnels without stalling. The key innovations are:

- **570x faster** on the 1-RTT hot path (99% of QUIC traffic)
- **97x faster** on Initial retransmits (cached HKDF keys)
- **Zero steady-state allocations** via sync.Pool and zero-alloc struct keys
- **N x fewer FDs** via UDP socket pool with DCID demux
- **200-600ms saved per site switch** via TCP warm-pool + first-N pre-warming through WireGuard
- **Zero DNS-lookup stalls** via prefetch + stale-while-revalidate + wildcard cache
- **No CLOSE-WAIT buildup** via immediate EOF propagation
- **CDN edge rotation handled** via sticky resolver + pool invalidation + sibling cleanup

Repository: github.com/cddeppe/xray-quiclink